

17장 포인터의 포인터

차례

1. 포인터의 포인터에 대한 이해
2. 다중 포인터 변수와 포인터의 필요성



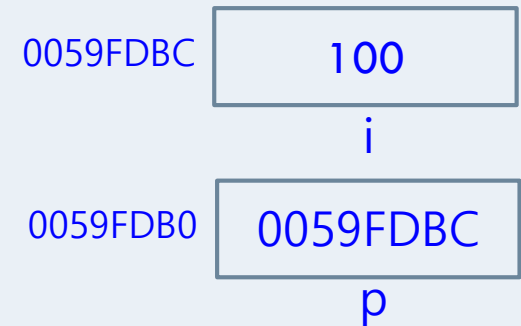
포인터도 변수임

3

- 포인터도 변수이므로 메모리 공간에 할당(8바이트)되고 주소를 가짐
- 주소연산자 &를 이용하여 포인터변수의 주소를 계산가능

```
#include <stdio.h>
int main(void)
{
    int i = 100;
    int* p = &i;
    printf("i의 주소=%p\n", &i);
    printf("i의 주소=%p\n", p);
    printf("p의 주소=%p\n", &p);
    return 0;
}
```

i의 주소=0059FDBC
i의 주소=0059FDBC
p의 주소=0059FDB0

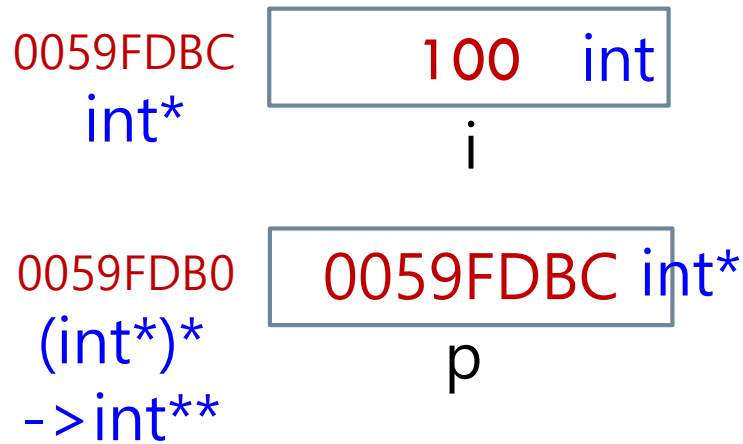




포인터의 주소의 자료형

4

- 변수 i의 자료형 -> int
- 포인터 p의 자료형 -> int*
- 포인터 p의 주소의 자료형 -> (int*)* -> int**





이중 포인터(double pointer)

5

- 포인터 변수의 주소를 저장하는 포인터 변수
- 포인터를 가리키는 포인터

```
자료형** 변수명;           // (자료형*)* 변수명;->새로운자료형* 변수명;  
  
int** dpi;                 // int*형 포인터의 주소를 저장하는 변수  
double** dpd;              // double*형 포인터의 주소를 저장하는 변수  
char** pdc;                // char*형 포인터의 주소를 저장하는 변수
```



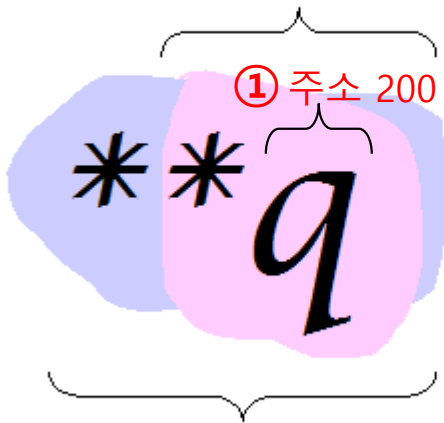
이중 포인터의 간접참조연산

6

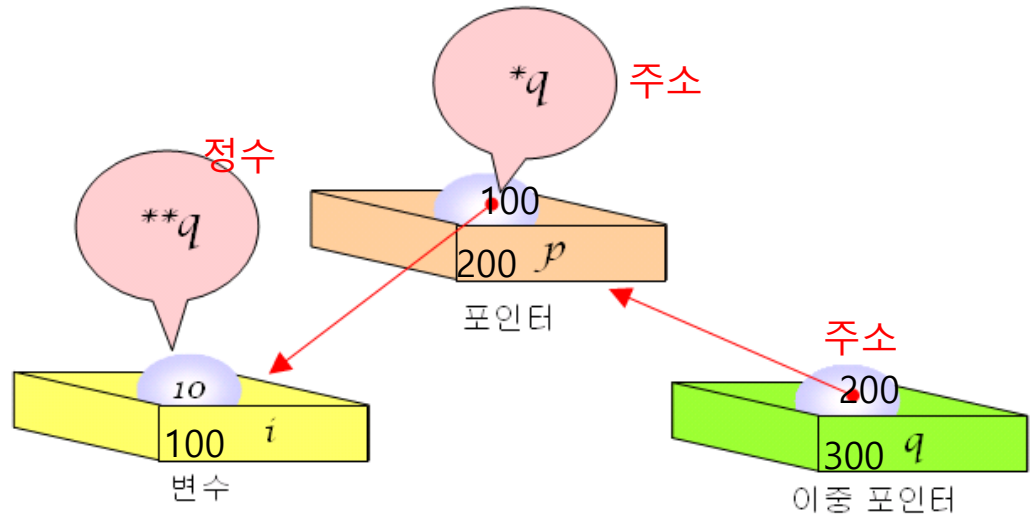
□ 이중 포인터(double pointer)의 간접 참조 연산자 해석

```
int i = 10;           // i는 int형 변수  
int* p = &i;          // p는 i를 가리키는 (단일)포인터  
int** q = &p;         // q는 포인터 p를 가리키는 이중 포인터
```

② q가 가리키는 위치의 내용 (주소 100)



③ *q가 가리키는 위치의 내용 (정수 10)





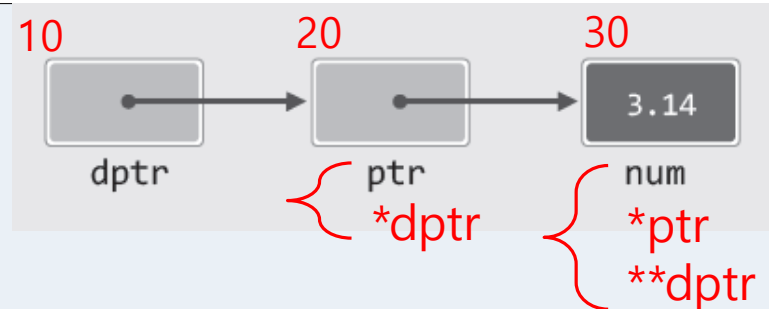
이중 포인터의 간접참조연산

7

이중 포인터 간단한 사용 예

```
double num = 3.14;  
double* ptr = &num;  
double** dptr = &ptr;
```

```
printf("%p %p \n", ptr, *dptr);  
printf("%g %g \n", num, **dptr);
```



// *dptr -> *(20) -> ptr -> &num -> 30

// **dptr -> *(*20) -> *(ptr) -> *30 -> num -> 3.14

예제 1

8

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int i = 100;
```

```
    int* p = &i;
```

```
    int** q = &p;
```

```
    *p = 200;
```

```
    printf("i=%d\n", i);
```

```
    **q = 300;
```

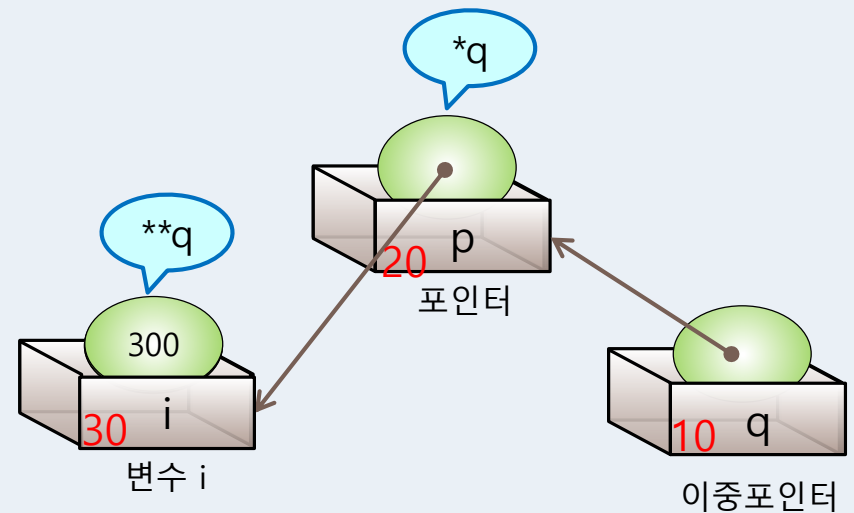
```
    printf("i=%d\n", i);
```

```
    return 0;
```

```
}
```

i=200

i=300



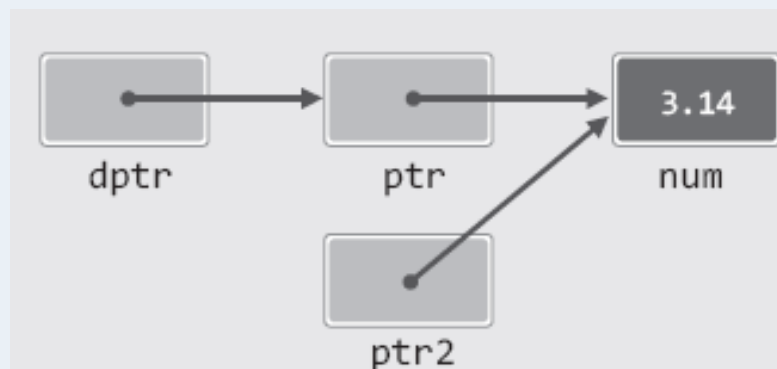
예제 2

9

```
#include <stdio.h>
int main(void)
{
    double num = 3.14;
    double* ptr = &num;
    double** dptr = &ptr;
    double* ptr2;

    printf("%9p %9p \n", ptr, *dptr);
    printf("%9g %9g \n", num, **dptr);
    ptr2 = *dptr;
    *ptr2 = 10.99;
    printf("%9g %9g \n", num, **dptr);
    return 0;
}
```

00D8FD08	00D8FD08
3.14	3.14
10.99	10.99





이중 포인터의 응용

10

- 다른 함수에 선언된 포인터의 값을 변경하는 경우
 - ▣ 주소에 의한 호출사용 -> 포인터의 주소를 받아야 함 -> 매개변수를 이중 포인터로 선언

```
int main(void)
{
    int num1 = 10, num2 = 20;
    int* ptr1 = &num1;
    int* ptr2 = &num2;
    ...
    Swap(&ptr1, &ptr2);
    ...
    //주소에 의한 호출
}
```

```
void Swap (int** p1, int** p2)
{
    int* temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}
```



예제 3 : 값에 의한 호출

11

```
#include <stdio.h>
void SwapIntPtr(int* p1, int* p2);
int main(void)
{
    int num1 = 10, num2 = 20;
    int* ptr1 = &num1;
    int* ptr2 = &num2;
    printf("*ptr1, *ptr2: %d %d \n", *ptr1, *ptr2);
    SwapIntPtr(ptr1, ptr2); // 값에 의한 호출(인자 int*, 매개변수 int*)
    printf("*ptr1, *ptr2: %d %d \n", *ptr1, *ptr2);
    return 0;
}
void SwapIntPtr(int* p1, int* p2)
{
    int* temp = p1;
    p1 = p2;
    p2 = temp;
}
```

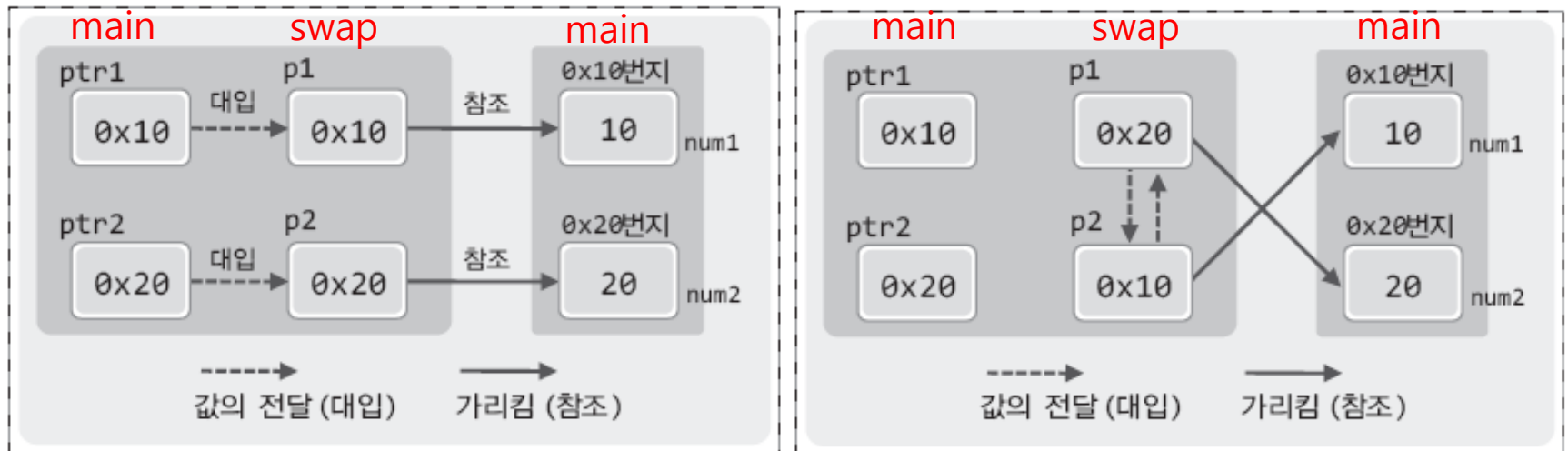
*ptr1, *ptr2: 10 20
*ptr1, *ptr2: 10 20



예제 3 : 값에 의한 호출

12

- 값에 의한 호출 사용시 매개변수의 값만 변경되고 메인 함수의 변수 값은 변경되지 않음





예제 4 : 주소에 의한 호출

13

```
#include <stdio.h>
void SwapIntPtr(int** dp1, int** dp2);
int main(void)
{
    int num1 = 10, num2 = 20;
    int* ptr1 = &num1;
    int* ptr2 = &num2;
    printf("*ptr1, *ptr2: %d %d \n", *ptr1, *ptr2);
    SwapIntPtr(&ptr1, &ptr2); // 주소에 의한 호출(인자 int*, 매개변수 int**)
    printf("*ptr1, *ptr2: %d %d \n", *ptr1, *ptr2);
    return 0;
}

void SwapIntPtr(int** dp1, int** dp2)
{
    int* temp = *dp1;
    *dp1 = *dp2;
    *dp2 = temp;
}
```

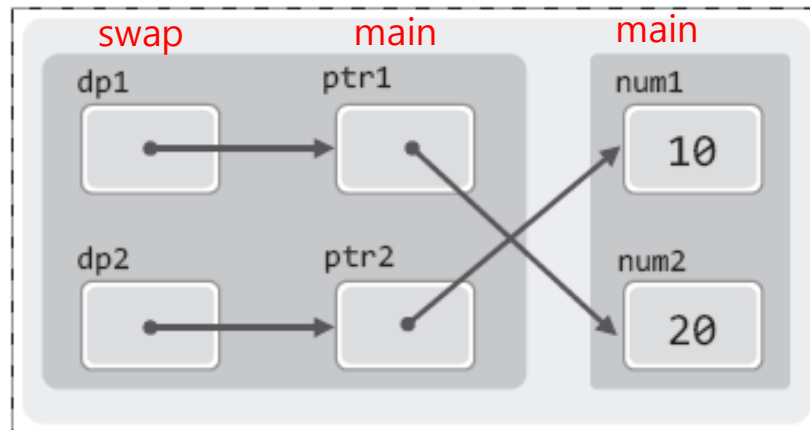
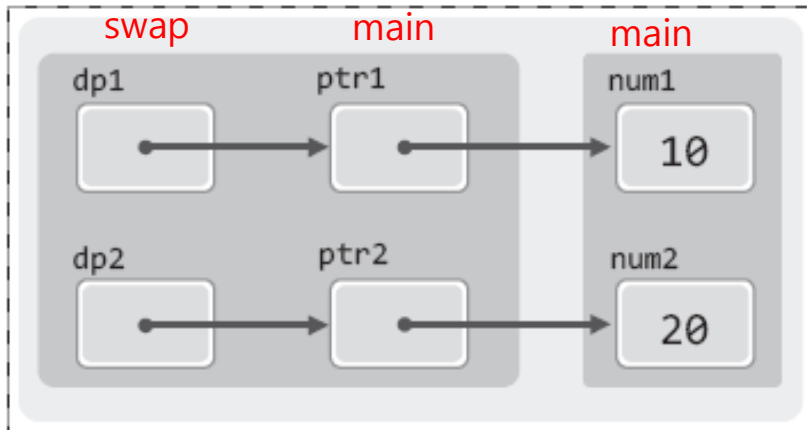
```
*ptr1, *ptr2: 10 20
*ptr1, *ptr2: 20 10
```



예제 4 : 주소에 의한 호출

14

- 주소에 의한 호출 사용시 매개변수의 값은 변경되지 않고 메인 함수의 변수값이 변경됨





이중 포인터의 응용2

15

- 함수의 인자에 포인터 배열을 받는 경우
 - ▣ 함수에서 배열을 받을 때는 주소에 의한 호출 -> 첫번째 원소가 포인터 -> 포인터의 주소를 받아야 함 -> 매개변수를 이중 포인터로 선언

```
int arr1[20];           // arr1의 자료형은 int*
double arr2[30];        // arr2의 자료형은 double*
int* arr3[20];          // arr3의 자료형은 int**
double* arr4[30];       // arr4의 포인터 형은 double**
```



예제 5 : 포인터 배열의 배열명

16

```
#include <stdio.h>
int main(void)
{
    int num1 = 10, num2 = 20, num3 = 30;
    int* ptr1 = &num1;
    int* ptr2 = &num2;
    int* ptr3 = &num3;
    int* ptrArr[ ] = { ptr1, ptr2, ptr3 };
    int** dptr = ptrArr;           // 포인터배열의 배열명은 포인터의 주소
    printf("%d %d %d \n", *(ptrArr[0]), *(ptrArr[1]), *(ptrArr[2]));
    printf("%d %d %d \n", *(dptr[0]), *(dptr[1]), *(dptr[2]));
    return 0;
}
```

10 20 30
10 20 30



예제 6 : 인자에 포인터 배열 전달

17

```
#include <stdio.h>
void print_number(int** dptr);
int main(void)
{
    int num1 = 10, num2 = 20, num3 = 30;
    int* ptrarr[3] = {&num1, &num2, &num3};
    print_number(ptrarr); // 주소에 의한 호출
    return 0;
}
void print_number(int** dptr)
{
    printf("numarray: %d %d %d \n", *(dptr[0]), *(dptr[1]), *(dptr[2]));
}
```

numarray: 10 20 30



이중 포인터를 가리키는 삼중 포인터

18

- 삼중 포인터 : 이중 포인터 변수의 주소 값을 저장하는 변수
- n중 포인터 : (n-1)중 포인터의 주소를 저장하는 변수

```
int*** tptr;
```



예제7 : 삼중포인터

19

```
#include <stdio.h>
int main(void)
{
    int num = 100;
    int* ptr = &num;
    int** dptr = &ptr;
    int*** tptr = &dptr;

    printf("%d %d\n", **dptr, ***tptr);

    return 0;
}
```

100 100



포인터의 필요성은 어디서 찾아야 하는가?

20

- scanf 함수와 같이 함수 내에서 함수 외부에 선언된 변수의 접근을 허용하기 위해서
- 메모리의 동적 할당 등등 PART 04에서 공부하는 내용을 통해서 포인터의 필요성을 다양하게 이해하게 됨
- 향후에 자료구조라는 과목을 공부하게 되면 보다 넓게 필요성을 이해할 수 있게 됨



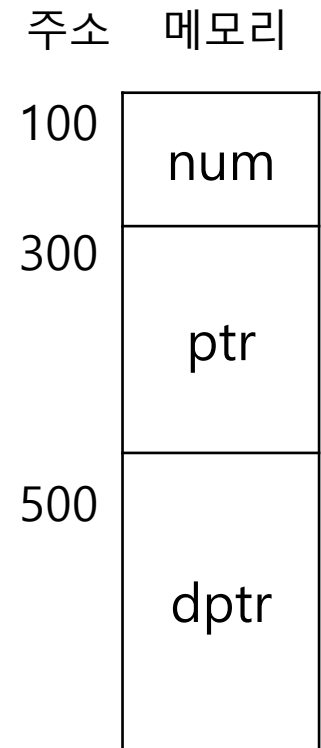
실습과제 1

21

- 아래의 변수가 우측 그림처럼 메모리가 할당될 때 다음 표의 빈칸을 채우시오.

```
double num = 6.28;  
double* ptr = &num;  
double** dptr = &ptr;
```

수식	결과값	결과값의 자료형
ptr		
dptr		
&ptr		
&dptr		
*ptr		
*dptr		
**dptr		



- 다음 코드에서 함수선언과 정의를 추가하여 결과처럼 나오도록 하시오.

```
#include <stdio.h>
// 함수선언
int main(void)
{
    int num1 = 50, num2 = 20, num3 = 30;
    int* ptrarr[3] = {&num1, &num2, &num3};
    int max;
    max = get_max(ptrarr, 3); // 함수호출
    printf("최댓값:%d\n", max);
    return 0;
}
// 함수정의
```

최댓값:50



실습과제3

23

- 다음 코드에서 함수선언과 정의를 추가하여 결과처럼 나오도록 하시오.

```
#include <stdio.h>
// 함수선언
int main(void)
{
    char* ptrarr[] = {"eagle", "tiger", "lion", "squirrel" };
    int count;
    count = sizeof(ptrarr)/sizeof(ptrarr[0]);
    prn_str(ptrarr, count);
    return 0;
}
// 함수정의
```

```
eagle
tiger
lion
squirrel
```

- 교재 368페이지의 문제 17-1을 푸시오. 소스코드의 모든 라인을 자세히 설명하시오. 메모리 그림을 이용하여 설명할 것



과제제출방법

25

- 아래 주소의 작성방법대로 github에 제출할 것
 - ▣ <https://github.com/2sungryul/cprogramming/tree/main>
- 기한 : 이클래스 과제 게시판의 마감시간 참조
- 소스코드의 첫 부분은 아래처럼 제목,날짜,작성자(학번,이름)를 작성할 것

```
// *****  
//   제   목   : 정수 4개의 평균을 구하는 프로그램  
//   날   짜   : 2023년 9월10일  
//   작성자   : 15010101 홍길동  
// *****  
  
// 소스코드 작성
```